SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA0503

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency.* (BL3, BL4)

Activity Tool : Industry Problem Solving Task (Medium)
Assessment Tool Weightage: 35%

Dr

QUESTIONS		Total Marks: 50	
Q.No	**Question**	**Bloom's Level**	**Marks**
1	An e-commerce company needs to store 10 million product records. Recommend a file organization strategy and justify your choice with cost analysis.	BL4 – Analyze	**5**
2	A banking system requires fast retrieval of transactions by account number and date range. Design an indexing strategy with appropriate index types.	BL4 – Analyze	**5**
3	A healthcare system stores patient records accessed both by PatientID and by doctor name. Design a multi-level indexing solution.	BL4 – Analyze	**5**
4	A logistics company needs to efficiently handle dynamic insertions and deletions of shipment records. Analyze whether B-Tree or B+ Tree is more suitable.	BL4 – Analyze	**5**
5	Design a hashing scheme for an HR system with 5000 employees. Evaluate static hashing vs. extendible hashing for your scenario.	BL3 – Apply	**5**
6	An airline reservation system must support point queries by flight number and range queries by date. Propose a combined index structure.	BL4 – Analyze	**5**
7	For a university student database, design the file organization and	BL4 – Analyze	**5**
	secondary indexing to support fast queries by department and CGPA range.		

8	Analyze the disk block utilization for Heap, Sorted, and Hashed file organizations given 100,000 records with 50 bytes each and 4KB blocks.	BL4 – Analyze	**5**
9	A social media platform needs to index 500 million posts by user_id, timestamp, and hashtag. Design a composite indexing strategy.	BL4 – Analyze	**5**
10	Compare the performance (insert, delete, search time) of Heap File, Sorted File, and Hashed File for a supermarket billing system with 1 million daily transactions.	BL4 – Analyze	**5**

Code of Conduct:

I I.JAWANTH(192525345) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: I.JASWANTH

RUBRICS FOR CALCULATION:

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of realworld problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution	25%	Innovative,	Logical and	Basic	Unclear or

Design		practical, and feasible solution aligned with industry standards	feasible solution with minor gaps	solution with limited feasibility	impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Answer:

Q1. E-commerce company – 10 million product records

Recommended file organization: Sorted File Organization

A sorted file stores product records in order of a key such as ProductID.

Why suitable?

- Product records can be searched efficiently using **binary search**.
- Sequential access is fast for product listings and reports.
- Indexes such as **B+ Tree** can further improve searching.
- Suitable for a very large dataset like **10 million records**.

Cost Analysis:

Assume:

- Records = 10,000,000
- Record size = 200 bytes Total storage:

If 4 KB blocks are used: **Comparison:**

Organization	Search	Insert	Storage
Heap	Slow	Fast	Low
Sorted	Fast	Costly	Low
Hashed	Very fast exact search	Fast	Moderate

Conclusion:

For an e-commerce system, **Sorted File + B+ Tree index on ProductID** is recommended because it provides fast retrieval and efficient range queries while keeping storage overhead reasonable.

Q2. Banking System – Indexing Strategy

A banking system needs fast retrieval of transactions using **AccountNumber** and **DateRange**.

Recommended strategy: Composite B+ Tree Index
Create an index on:
Example:
CREATE INDEX idx_account_date
ON Transactions(AccountNumber, TransactionDate); **Why B+ Tree?**

- Fast search using AccountNumber.
- Efficient range search using dates.
- Supports both exact and range queries.
- Suitable for large transaction tables.

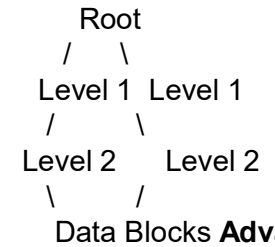
Example query:
SELECT *
FROM Transactions
WHERE AccountNumber = 10101
AND TransactionDate BETWEEN '2026-01-01' AND '2026-01-31'; **Conclusion:**
A **composite B+ Tree index on (AccountNumber, TransactionDate)** provides efficient transaction retrieval and date-range searching.

Q3. Healthcare System – Multi-Level Indexing
Patient records are accessed using **PatientID** and **DoctorName**.
Recommended design:

- Primary index on PatientID • Secondary index on DoctorName
- Use **B+ Tree** for both indexes.

Patient Table
|
+---- B+ Tree Index → PatientID
|
+---- B+ Tree Index → DoctorName
Example:
CREATE INDEX idx_patient
ON Patients(PatientID);

CREATE INDEX idx_doctor ON
Patients(DoctorName); **Multi-level structure:**



Advantages:

- Fast PatientID search.
- Fast doctor-based search.
- Multi-level indexing reduces disk accesses.
- B+ Tree supports large healthcare databases efficiently.

Conclusion:
Using **primary and secondary B+ Tree indexes** provides fast access through both PatientID and DoctorName.

Q4. Logistics – B-Tree vs B+ Tree
The logistics company performs frequent **insertions and deletions** of shipment records.

Feature	B-Tree	B+ Tree
Search	Fast	Very fast
Insert	Fast	Fast
Delete	Fast	Fast
Range queries	Good	Excellent
Sequential access	Moderate	Excellent
Data storage	Internal + leaf nodes	Mainly leaf nodes

B+ Tree is recommended.

Reasons:

1. All actual records are stored at leaf level.
2. Leaf nodes are linked.
3. Range queries are efficient.
4. Tree remains balanced after insert/delete.
5. Fewer disk accesses are required.

Conclusion:

For dynamic shipment records, **B+ Tree is more suitable** because it provides efficient insertion, deletion, searching and especially range queries.

Q5. HR System – 5000 Employees A hashing scheme can use:

For example:

Static Hashing

EmployeeID
↓
Hash Function
↓
Bucket
↓
Employee Record **Advantages:**

- Simple implementation.
- Very fast exact search.
- Suitable when the number of records is relatively stable.

Disadvantages:

- Fixed number of buckets.
- Overflow can occur when employees increase.

Extendible Hashing

Extendible hashing dynamically increases buckets when required.

Advantages:

- Handles employee growth.
- Reduces overflow.
- Dynamic bucket splitting.
- Good for frequently changing databases.

Conclusion:

For 5000 employees with expected future growth, **Extendible Hashing is preferred** over static hashing.

Q6. Airline Reservation – Combined Index The system requires:

- Point queries by **FlightNumber**
- Range queries by **Date** Use two indexes:

Flight Table

|
+---- B+ Tree → FlightNumber
|
+---- B+ Tree → FlightDate Example:

```
CREATE INDEX idx_flight_no
ON Flights(FlightNumber);

CREATE INDEX idx_flight_date ON
Flights(FlightDate);
```

FlightNumber index:

- Provides fast exact search.

FlightDate index:

- Provides efficient date-range queries.

Example:

```
SELECT *
FROM Flights
WHERE FlightNumber = 'AI202';
```

Range query:

```
SELECT *
FROM Flights
WHERE FlightDate BETWEEN '2026-08-01' AND '2026-08-10';
```

Conclusion:
Using **two B+ Tree indexes** provides efficient point and range queries.

Q7. University Database – Department and CGPA
The university database should support queries by **Department** and **CGPA range**.
File Organization
Use **Heap File Organization** because student records are frequently inserted and updated.
Secondary Indexes Create:

```
CREATE INDEX idx_department
ON Students(Department);

CREATE INDEX idx_cgpa
ON Students(CGPA);
```

For CGPA range queries, a **B+ Tree** is suitable.
Students
|
+---- Secondary B+ Tree → Department
|
+---- Secondary B+ Tree → CGPA Example:

```
SELECT *
FROM Students
WHERE Department = 'CSE';
SELECT *
FROM Students
WHERE CGPA BETWEEN 8.0 AND 9.0;
```

Conclusion:
Use **Heap File + secondary indexes**, with a B+ Tree for CGPA range searching.

Q8. Disk Block Utilization Given:

- Records = 100,000
- Record size = 50 bytes
- Block size = 4 KB = 4096 bytes **Records per block**

So, **81 records per block**.
Number of blocks
Space used

Allocated space
Unused space
Comparison

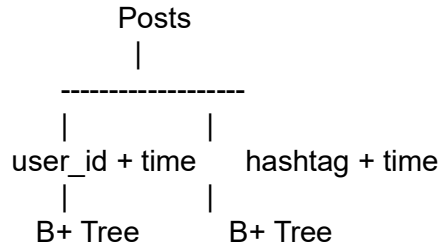
File Organization	Search	Insert	Range Query
Heap	Slow	Very fast	Slow
Sorted	Fast	Slow	Very fast
Hashed	Very fast	Fast	Poor

Conclusion:
All three require approximately **1235 data blocks**, but their performance differs based on the type of query.

Q9. Social Media – 500 Million Posts The system needs indexing on:

- user_id
- timestamp
- hashtag

Because there are **500 million posts**, indexes must be carefully designed.
Recommended indexes
1. User + Timestamp
CREATE INDEX idx_user_time ON Posts(user_id, timestamp); Useful for:
User → Recent posts
2. Hashtag + Timestamp
CREATE INDEX idx_hash_time ON Posts(hashtag, timestamp); Useful for:
Hashtag → Posts within date/time range
Structure



Advantages:

- Fast user-based retrieval.
- Efficient chronological retrieval.
- Fast hashtag searches.
- B+ Trees support range queries.
- Composite indexes reduce unnecessary table scanning.

Conclusion:
For 500 million posts, use **composite B+ Tree indexes on (user_id, timestamp) and (hashtag, timestamp)**.

Q10. Supermarket Billing – 1 Million Transactions Compare Heap, Sorted and Hashed file organizations.

Operation	Heap File	Sorted File	Hashed File

Insert	Very Fast	Slow	Fast
Delete	Moderate	Slow	Fast
Exact Search	Slow	Fast	Very Fast
Range Search	Slow	Very Fast	Poor
Maintenance	Easy	High	Moderate

Heap File
New transactions are appended at the end.
Insert: approximately **Search:**
Sorted File
Records are maintained in sorted order.
Search: with suitable search/indexing
Insert/Delete: Expensive because records may need rearrangement.
Hashed File
A hash function determines the storage bucket.
Exact search: approximately average
Range search: Poor
Recommendation
For a supermarket billing system with **1 million daily transactions**, use:
Hashed File + B+ Tree secondary index

Transaction
|
+---- Hash → TransactionID
|
+---- B+ Tree → Date

This gives fast transaction lookup while the B+ Tree supports date-based reports and range queries.
Final conclusion:
Hashed organization is best for frequent exact transaction searches, while a **B+ Tree secondary index** should be added for date/range queries.